

Compressing Chain-of-Thought in Transformer-based Parity Function Learning

ZHANG GAOLE

Department of Systems Science
Graduate School of Informatics, Kyoto University
Shimodaira-Honda Lab

Supervisor: Professor Hidetoshi Shimodaira

August 7, 2025

Outline

- 1 Introduction & Motivation
- 2 Experimental Approach
- 3 Results
- 4 Discussion & Conclusion

The Parity Problem: A Canonical Challenge

Task

The k -**Parity** function determines if a binary sequence, composed by d -bits of $-1, 1$, has an even number of “ -1 ”s on fixed k active positions.

$$\chi_S(x) = \prod_{i \in S} x_i, \quad x \in \{\pm 1\}^d, \quad S \subseteq [d] = \{1, \dots, d\}, \quad |S| = k$$

If number of “ -1 ”s is even, it outputs 1, otherwise outputs -1 .

Problem

Despite its simplicity, Transformers trained end-to-end **consistently fail** to learn Parity. This is a well-documented limitation.

Why Parity Hard for Transformers learning End-to-End?

Two Main Theories: 1. High Sensitivity

Parity is highly sensitive. Flipping any single input bit flips the output. But Transformers have an inductive bias for low-sensitivity functions [Vasudeva et al.(2025) Vasudeva, Fu, Zhou, Kau, Huang, and Sharan, Hahn and Rofin(2024)].

$$s(x, f) := \frac{1}{4} \sum_{i=1}^d |f(x) - f(x^{\oplus i})|^2, \quad as_d(f) := \frac{1}{2^d} \sum_{x \in \{\pm 1\}^d} s(x, f)$$

where $x^{\oplus i}$ denotes the bitstring x with the i -th bit flipped.
For k -Parity, $as_d(\chi_S) = |S| = k$, linearly increasing with k .

Why Parity Hard for Transformers learning End-to-End?

Two Main Theories: 2. Uninformative Gradients

Parity is an orthogonal and normalized basis of Boolean function space. Suppose the hypothesis class H is all k -parity functions on d -bits input.

$$\bar{L}_{\chi_T}(w) = E_x \|f_w(x) - \chi_T(x)\|_2^2, \quad f_w(x) = \sum_{S \subseteq [d]} \tilde{f}_w(S) \chi_S(x)$$

$$\begin{aligned} & \text{Var}_{\chi_T \in H} [\nabla_w \bar{L}_{\chi_T}(w)] \\ & \leq 4 E_{\chi_T \in H} \|\nabla_w \tilde{f}_w(T)\|_2^2 \\ & \approx \frac{E_x \|\nabla_w f_w(x)\|_2^2}{|H|} \leq \frac{\text{poly}(d)}{2^d} \approx 0 \end{aligned}$$

where x and χ_T are uniformly distributed. Then we can bound the loss gradient over H in a small range and establish adversarial oracle answering uninformative mean gradient with high

probability[Wies et al.(2023)Wies, Levine, and Shashua, Kim and Suzuki(2025), Shalev-Shwartz et al.(2017)Shalev-Shwartz, Shamir, and Shammah].

Chain-of-Thought (CoT) Makes Parity Learnable

The Solution: Decompose the Problem

Instead of end-to-end learning, we provide intermediate supervision (teacher forcing) using a **Chain-of-Thought (CoT)**.

- The model generates a sequence of intermediate steps.
- This transforms one high-sensitivity problem into a sequence of simple, low-sensitivity sub-problems (e.g., parity of just two bits).
- Each step has a smooth loss landscape, making it easily learnable.

Chain-of-Thought (CoT) Makes Parity Learnable

The Solution: Decompose the Problem

Instead of end-to-end learning, we provide intermediate supervision (teacher forcing) using a **Chain-of-Thought (CoT)**.

- The model generates a sequence of intermediate steps.
- This transforms one high-sensitivity problem into a sequence of simple, low-sensitivity sub-problems (e.g., parity of just two bits).
- Each step has a smooth loss landscape, making it easily learnable.

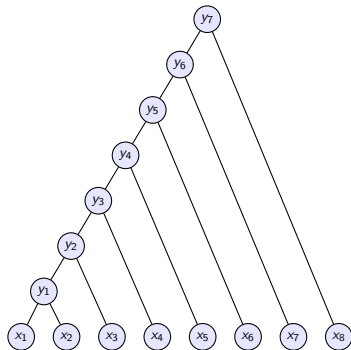
Core Research Question

Prior work shows CoT makes Parity learnable. But...

If compressing CoT by computational depth or reasoning steps influence learning effectiveness?

Different Computational Depth in CoT (1): Skewed Tree

One way to structure the computation is a deep, sequential chain.

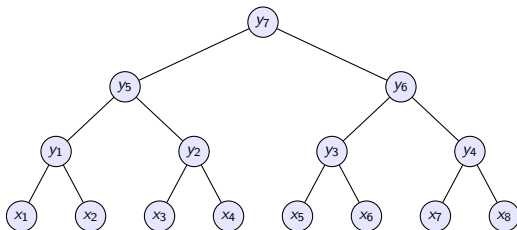


Properties

- **Deep:** The computational depth is linear, $O(k)$.
- **Sequential:** Each (y_i) depends only on the previous result (y_{i-1}) and the next input (x_{i+1}) .

Different Computational Depth in CoT (2): Balanced Tree

Another way is a shallow, parallel hierarchy.



Properties

- **Shallow:** The computational depth is logarithmic, $O(\log k)$.
- **Parallel:** Operations in each layer can be done in parallel.

① **Analyze CoT Structure Trade-offs:**

Empirically investigate the trade-off between convergence speed and final accuracy across 7 CoT variants.

② **Propose Log CoT for Test Performance:**

Introduce *Log CoT* to compress reasoning steps to $O(\log k)$, improving test performance against error accumulation.

③ **Identify Key Factors for Performance:**

Empirically conclude that compressing CoT depth slows training, while compressing CoT steps improves test performance, with Log CoT we can get better test performance with longer training.

CoT Structures Under Investigation

1. Linear CoT (Skewed)

Based on the skewed tree. Results in a long $O(k)$ computational depth and steps. Also, to investigate the influence of positional dependency, i.e. different distributions of the distance between “child” and “parent” tokens, we employ 3 variance of CoT ordering, i.e., standard (forward), reversed and random ordering.

CoT Structures Under Investigation

1. Linear CoT (Skewed)

Based on the skewed tree. Results in a long $O(k)$ computational depth and steps. Also, to investigate the influence of positional dependency, i.e. different distributions of the distance between “child” and “parent” tokens, we employ 3 variance of CoT ordering, i.e., standard (forward), reversed and random ordering.

2. Linear CoT (Balanced)

Based on the balanced tree. Also results in an $O(k)$ step chain, but with $O(\log k)$ depth. Samely, we have standard, reversed and random ordering.

CoT Structures Under Investigation

1. Linear CoT (Skewed)

Based on the skewed tree. Results in a long $O(k)$ computational depth and steps. Also, to investigate the influence of positional dependency, i.e. different distributions of the distance between “child” and “parent” tokens, we employ 3 variance of CoT ordering, i.e., standard (forward), reversed and random ordering.

2. Linear CoT (Balanced)

Based on the balanced tree. Also results in an $O(k)$ step chain, but with $O(\log k)$ depth. Samely, we have standard, reversed and random ordering.

3. Log CoT (Proposed Method)

Based on the balanced tree, but compresses each layer into a single token. Results in a short $O(\log k)$ step chain with $O(k)$ embedding size.

1: Linear CoT (Skewed-standard)

Linear CoT (Skewed-standard) trained with teacher forcing															
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	$\hat{y}_1$	$\hat{y}_1 \approx y_1 = x_1 * x_8$					h=0	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	$\hat{y}_2$	$\hat{y}_2 \approx y_2 = y_1 * x_2$				h=1	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	$\hat{y}_3$				h=2	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	$\hat{y}_4$			h=3	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	$\hat{y}_5$		h=4	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	$\hat{y}_6$	h=5	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	y_6	$\hat{y}_7$	h=6
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	y_6	y_7	Emb_dim=1
$(x_1, x_2, \dots, x_d), x_i \in \{\pm 1\}$ Input length: $O(k)$								$(y_1, y_2, \dots, y_{k-1}), y_i \in \{\pm 1\}, \hat{y}_i \approx y_i$ CoT : $O(k)$ steps, $O(k)$ depth							

here we set $d = k = 8$, k is active position size.

1: Linear CoT (Skewed-reversed)

Linear CoT (Skewed-reversed) trained with teacher forcing															
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	$\hat{y}_1$	$\hat{y}_1 \approx y_1 = x_7 * x_8$					h=0	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	$\hat{y}_2$	$\hat{y}_2 \approx y_2 = y_1 * x_6$				h=1	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	$\hat{y}_3$				h=2	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	$\hat{y}_4$			h=3	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	$\hat{y}_5$		h=4	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	$\hat{y}_6$	h=5	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	y_6	$\hat{y}_7$	h=6
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	y_6	y_7	Emb_dim=1
$(x_1, x_2, \dots, x_d), x_i \in \{\pm 1\}$ Input length: $O(k)$								$(y_1, y_2, \dots, y_{k-1}), y_i \in \{\pm 1\}, \hat{y}_i \approx y_i$ CoT : $O(k)$ steps, $O(k)$ depth							

here we set $d = k = 8$, k is active position size.

1: Linear CoT (Skewed-random)

Linear CoT (Skewed-random) trained with teacher forcing															
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	$\hat{y}_1$	$\hat{y}_1 \approx y_1 = x_6 * x_8$					h=0	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	$\hat{y}_2$	$\hat{y}_2 \approx y_2 = y_1 * x_3$				h=1	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	$\hat{y}_3$				h=2	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	$\hat{y}_4$			h=3	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	$\hat{y}_5$		h=4	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	$\hat{y}_6$	h=5	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	y_6	$\hat{y}_7$	h=6
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	y_6	y_7	Emb_dim=1
$(x_1, x_2, \dots, x_d), x_i \in \{\pm 1\}$ Input length: $O(k)$								$(y_1, y_2, \dots, y_{k-1}), y_i \in \{\pm 1\}, \hat{y}_i \approx y_i$ CoT : $O(k)$ steps, $O(k)$ depth							

here we set $d = k = 8$, k is active position size.

2: Linear CoT (Balanced-standard)

Linear CoT (Balanced-standard) trained with teacher forcing															
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	$\hat{y}_1$	$\hat{y}_1 \approx y_1 = x_1 * x_2$						h=0
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	$\hat{y}_2$	$\hat{y}_2 \approx y_2 = x_3 * x_4$					
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	$\hat{y}_3$	$\vdots$				
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	$\hat{y}_4$	$\vdots$			
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	$\hat{y}_5$	h=1		
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5			$\hat{y}_6$
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	y_6	$\hat{y}_7$	h=3
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	y_6	y_7	Emb_dim=1
$(x_1, x_2, \dots, x_d), x_i \in \{\pm 1\}$ Input length: $O(k)$								$(y_1, y_2, \dots, y_{k-1}), y_i \in \{\pm 1\}, \hat{y}_i \approx y_i$ CoT : $O(k)$ steps, $O(\log k)$ depth							

here we set $d = k = 8$, k is active position size.

2: Linear CoT (Balanced-reversed)

Linear CoT (Balanced-reversed) trained with teacher forcing															
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	$\hat{y}_1$	$\hat{y}_1 \approx y_1 = x_1 * x_2$					h=0	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	$\hat{y}_2$	$\hat{y}_2 \approx y_2 = x_3 * x_4$					
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	$\hat{y}_3$					
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	$\hat{y}_4$				
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	$\hat{y}_5$	h=1		
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5			$\hat{y}_6$
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	y_6	$\hat{y}_7$	h=3
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	y_6	y_7	Emb_dim=1
$(x_1, x_2, \dots, x_d), x_i \in \{\pm 1\}$ Input length: $O(k)$								$(y_1, y_2, \dots, y_{k-1}), y_i \in \{\pm 1\}, \hat{y}_i \approx y_i$ CoT : $O(k)$ steps, $O(\log k)$ depth							

here we set $d = k = 8$, k is active position size.

2: Linear CoT (Balanced-random)

Linear CoT (Balanced-random) trained with teacher forcing															
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	$\hat{y}_1$	$\hat{y}_1 \approx y_1 = x_1 * x_5$					h=0	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	$\hat{y}_2$	$\hat{y}_2 \approx y_2 = x_3 * x_4$					
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	$\hat{y}_3$					
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	$\hat{y}_4$				
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	$\hat{y}_5$	h=1		
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	$\hat{y}_6$	h=3	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	y_6		
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	y_6	$\hat{y}_7$	
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	y_1	y_2	y_3	y_4	y_5	y_6	y_7	Emb_dim=1
$(x_1, x_2, \dots, x_d), x_i \in \{\pm 1\}$ Input length: $O(k)$								$(y_1, y_2, \dots, y_{k-1}), y_i \in \{\pm 1\}, \hat{y}_i \approx y_i$ CoT : $O(k)$ steps, $O(\log k)$ depth							

here we set $d = k = 8$, k is active position size.

3: Log CoT (Proposed)

Log CoT trained with teacher forcing (Proposed)											
e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8	$\hat{z}_1$			$h=0$
e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8	z_1	$\hat{z}_2$		$h=1$
e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8	z_1	z_2	$\hat{z}_3$	$h=2$
e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8	z_1	z_2	z_3	Emb_dim $\frac{k}{2} = O(k)$ because z_1 needs $\frac{k}{2}$ dims to encoding the first hierarchy.
$\begin{bmatrix} x_1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} x_2 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} x_3 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} x_4 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} x_5 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} x_6 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} x_7 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} x_8 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} x_1x_2 \\ x_3x_4 \\ x_5x_6 \\ x_7x_8 \end{bmatrix}$	$\begin{bmatrix} y_1y_2 \\ y_3y_4 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} y_5y_6 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	
								$= \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix}$	$= \begin{bmatrix} y_5 \\ y_6 \\ 0 \\ 0 \end{bmatrix}$	$= \begin{bmatrix} y_7 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	
$e_i = [x_i, 0, \dots, 0] \in R^{\binom{k}{2}}, x_i \in \{\pm 1\}, i \in [d]$								$z_i \in R^{\binom{k}{2}}, y_i \in \{\pm 1\}, \hat{z}_i \approx z_i$			
Input length: $O(k)$								CoT : $O(\log k)$ steps and depth			

here we set $d = k = 8$, k is active position size.

Experimental Setup

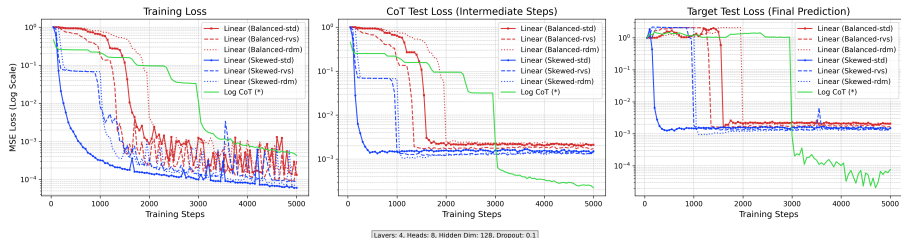
Model

- Decoder-only Transformer (PyTorch).
- 4 Layers, 8 Attention Heads, Hidden Dim 128.
- Initial input $x \in \{\pm 1\}^d$, $d = 64$ for all experiments.

Training and Evaluation

- **Task:** Generate full CoT for k -Parity ($k = 16, 32, 64$).
- **Sample:** [initial input x ; CoT y].
- **Supervision:** Teacher-forcing with Mean Squared Error (MSE) loss.
- **Metrics:** We measure loss on the CoT (process) and the final answer (outcome) separately.

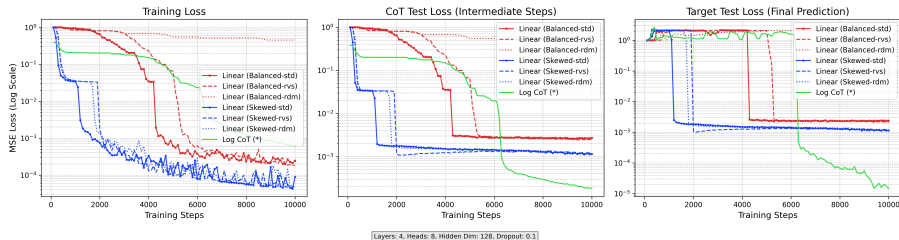
Results (k=16): Small Scale Speed Ranking



Observation

- **Speed Ranking:** Skewed family (blue) > Balanced family (red) > Log CoT (green).
- **Conclusion:** At small scale, both Linear(skewed) family and Linear(balanced) family obviously faster than Log CoT, but Log CoT gets the best final test performance.

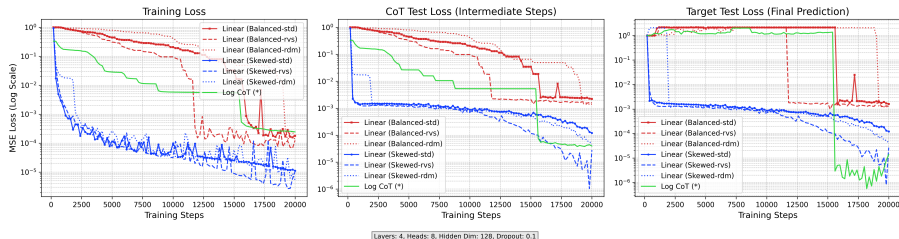
Results (k=32): Scalability Issues of the Balanced Family



Observation

- **Speed Ranking:** Skewed family (blue) still convergent fast. Balanced family (red) speed degrades, closed to Log CoT.
- **Conclusion:** Log CoT (green), though slow, but still achieves the best final test performance.

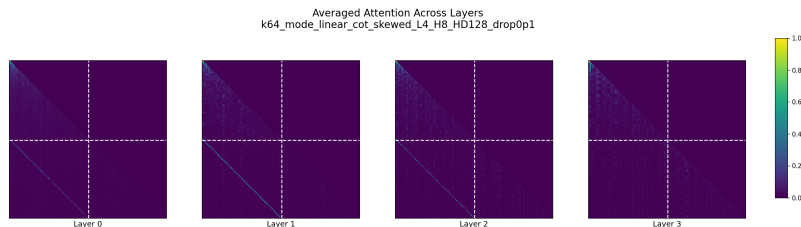
Results (k=64): The Final Trade-off



Observation

- **Speed Ranking:** Skewed family (blue) remains the fastest. Balanced family (red) are generally slower than Log CoT.
- **Conclusion:** Log CoT (green) is the winner on final test performance.

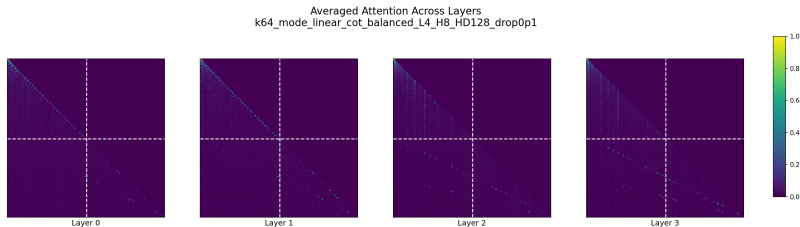
Analysis: Attention Pattern of Skewed CoT ($k=64$)



Insight

- **Pattern:** A clear, local attention pattern forms in the very first layers (Layer 0, 1).
- **Interpretation:** The learning task is simple; the model quickly identifies the "attend to previous token" rule.

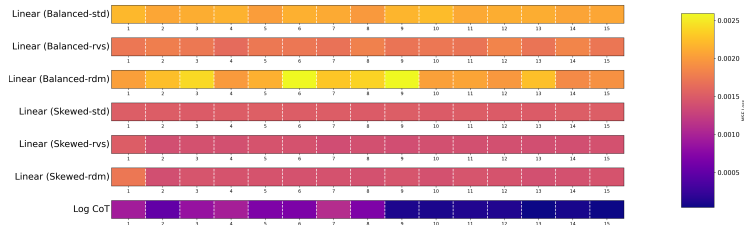
Analysis: Attention Pattern of Balanced CoT (k=64)



Insight

- **Pattern:** Attention remains diffuse in early layers. A meaningful pattern only emerges in deeper layers.
- **Interpretation:** The model requires more computational depth to resolve complex non-local dependencies, explaining its slow convergence.

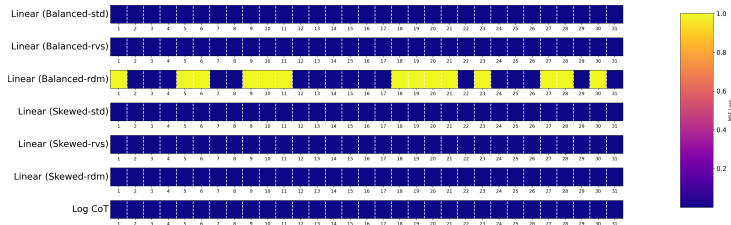
Analysis: Detailing Error Accumulation on Each CoT Bit ($k=16$)



Insight

- Long-chain CoTs (linear models) show escalating error (lighter colors).
- Log CoT's short chain maintains low error, explaining its high accuracy.

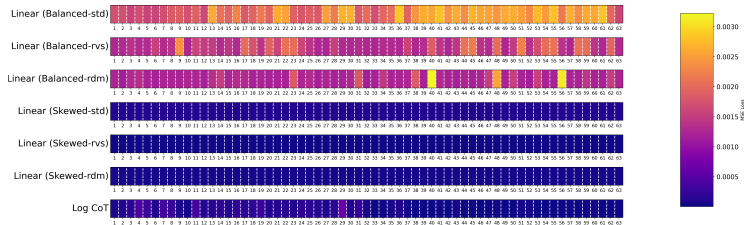
Analysis: Detailing Error Accumulation on Each CoT Bit ($k=32$)



Insight

- Long-chain CoTs (linear models) show escalating error (lighter colors).
- Log CoT's short chain maintains low error, explaining its high accuracy.

Analysis: Error Accumulation on Each CoT Bit ($k=64$)



Insight

- Long-chain CoTs (linear models) show escalating error (lighter colors).
- Log CoT's short chain maintains low error, explaining its high accuracy.

Synthesis: Comparing CoT Architectures

CoT Structure	CoT steps	Emb Size	Convergent Steps
Linear (skewed)	$O(k)$	$O(1)$	$O(1)$
Linear (balanced)	$O(k)$	$O(1)$	increases with k
Log CoT	$O(\log k)$	$O(k)$	$\approx$ Linear (balanced)

Synthesis: Key Takeaways

Compressing Depth Slows Convergence

- Simple, local dependencies (Skewed) converge fastest. Performance is robust to ordering changes.
- Non-local dependencies (Balanced) are brittle and do not scale.

Compressing Steps Improves Test Accuracy

- Long $O(k)$ reasoning chains suffer from error accumulation.
- Log CoT's short $O(\log k)$ chain is robust, leading to the highest final accuracy.

Log CoT: A Principle of Computational Compression

The Core Idea of Log CoT

Trade sequential reasoning length for representational width. Use the model's high-dimensional embedding space as a **parallel computational register**.

Traditional CoT:

- 1 step = 1 scalar operation
- Parallel work is serialized
- Long chains → error prone

Log CoT:

- 1 step = 1 layer of operations
- Parallel work is vectorized
- Short chains → robust

Conclusion and Future Work

Conclusion

CoT structure involves a direct trade-off between speed and accuracy. Our proposed Log CoT, while slower to train, achieves superior accuracy on complex problems by creating a robust, short reasoning chain.

Limitations and Future Work

- **Task Generality:** Apply Log CoT to more complex algorithmic tasks.
- **Relaxing Supervision:** Investigate learnability without perfect teacher-forcing.
- **Scaling Laws:** Explore the interaction between model size, k , and the speed-accuracy trade-off.

Thank You Very Much

ZHANG GAOLE

Department of Systems Science, Kyoto University

References I



M. Hahn and M. Rofin.

Why are sensitive functions hard for transformers?, 2024.

URL <https://arxiv.org/abs/2402.09963>.



J. Kim and T. Suzuki.

Transformers provably solve parity efficiently with chain of thought.

In *The Thirteenth International Conference on Learning Representations*, 2025.

URL <https://openreview.net/forum?id=n2NidsYDop>.



S. Shalev-Shwartz, O. Shamir, and S. Shammah.

Failures of gradient-based deep learning.

In *International Conference on Machine Learning*, 2017.



B. Vasudeva, D. Fu, T. Zhou, E. Kau, Y. Huang, and V. Sharan.

Transformers learn low sensitivity functions: Investigations and implications.

In *The Thirteenth International Conference on Learning Representations*, 2025.

URL <https://openreview.net/forum?id=4ikjWBs3tE>.



N. Wies, Y. Levine, and A. Shashua.

Sub-task decomposition enables learning in sequence to sequence tasks.

In *International Conference on Learning Representations*, 2023.